

The OpenStack compute service, known as **Nova**, acts as the "conductor" for the virtual machine (VM) lifecycle, managing tasks such as provisioning, starting, stopping, and decommissioning instances 1, 2. Its architecture relies on several specialized components that coordinate through a central messaging system.

Core Components of Nova

- **nova-api:** This service is the **primary entry point** for all user requests 2. It interacts with user API calls to manage compute instances and adheres to the **REST architectural style**, handling standard HTTP requests like GET, POST, PUT, and DELETE 2-4. It communicates with other compute components exclusively over the **message bus** 5.
- **nova-scheduler:** Often called the **"brain"** of VM placement, this service listens for new instance requests on the message bus 5, 6. Its core function is to **select the most suitable compute node** to launch a new VM by analyzing factor like resource availability (CPU, RAM, disk space), user preferences, and placement groups for redundancy 5, 7, 8.
- **nova-conductor:** This service is a critical **security and scalability element** 9. It acts as a secure **intermediary** between compute nodes and the OpenStack database 9, 10. By offloading database tasks, it ensures that even if a physical host is compromised, the attacker does not gain direct access to the central database 9, 11.
- **nova-compute:** This is the process responsible for the **actual starting and terminating of virtual machines** 5. It runs directly on the compute nodes (the physical servers) and listens for instructions over the message bus 5. To maintain security, these nodes are **denied direct access to the database** 5, 12.

Communication and Data Management

- **Message Queue (Message Bus) Role:** The message bus is the **communication backbone** of the compute service. Components like nova-api, nova-scheduler, and nova-compute do not talk to each other directly; instead, they send and receive messages over this bus to coordinate complex tasks asynchronously 5.
- **Database Interaction:** The OpenStack database stores critical cloud state information. While services like the nova-api and nova-scheduler may interact with it, the **compute nodes (nova-compute) are isolated from it** 5, 9. The nova-conductor fields database queries on their behalf, providing a layer of protection and allowing for **horizontal scaling**—meaning you can deploy multiple conductors on separate machines to handle increased workloads 6, 11.

Exam-Focused Structured Notes

1. **Component Functions:**
2. **nova-api:** Entry point; handles RESTful HTTP requests 2, 4.
3. **nova-scheduler:** Selection logic; uses **Filtering** (removing unsuitable hosts) and **Weighting** (ranking remaining hosts) to choose a node 13, 14.
4. **nova-conductor:** Security proxy; shields the database from compute nodes 9, 10.

5. **nova-compute:** Executioner; manages the VM lifecycle on the physical hardware 5.
6. **Key Security Rules:**
7. **Isolation:** Compute nodes must never have direct database access 5.
8. **Deployment:** nova-conductor and nova-compute **must not be deployed on the same physical machine**, or the security benefits are nullified 6, 11.
9. **Scheduling Process (3 Steps):**
10. **Filtering:** Eliminate nodes that don't meet requirements (e.g., lack of RAM or specific hardware like GPUs) 13.
11. **Weighting:** Rank remaining nodes based on priorities like lowest utilization or network proximity 13, 14.
12. **Selection:** Choose the node with the highest weight 14.
13. **Scalability:** The architecture supports **horizontal scaling**, particularly for the conductor and scheduler, allowing the cloud to grow by adding more service instances across multiple machines 11, 15.
14. **Inter-Service Integration:** Nova relies on **Keystone** for authentication, **Glance** for VM images, and **Neutron** for networking connectivity 16-18.